ECE 486: Final Project Report

Spring 2024

Final Project Report

TA: Yu-Ju Chang, Section B

Dash Kamriani Beard, Aniketh Rayudu

Introduction

For our ECE 486 final project, we created a controller to invert and stabilize a pendulum. Our setup is called a Reaction Wheel Pendulum (RWP): a metal pendulum connected to a free-rotating mount at one end and a motorized reaction wheel at the other. We drive the motor to control the reaction wheel's speed, while two sensors that measure rotor angle and pendulum angle allow us to control and determine the orientation of our system at any point in time. Our controller moves the system from stable equilibrium, where the pendulum hangs downward, to inverted unstable equilibrium by swinging the pendulum side to side with the spinning rotor.

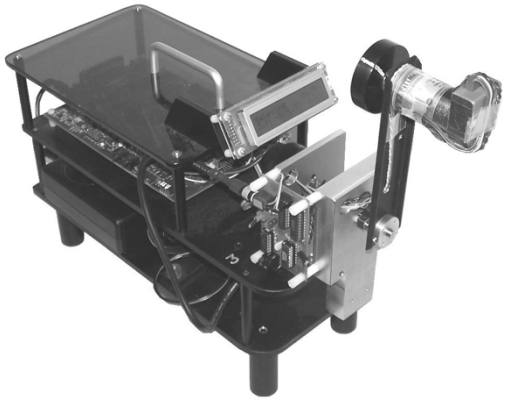


Figure 1: The Reaction Wheel Pendulum

The spinning of the rotor moves our pendulum arm due to Newton's Third Law: to spin the rotor, our motor applies a torque to it. Because the rotor is attached to the free-swinging pendulum, the pendulum experiences an equal and opposite torque that swings it in the direction opposite of rotor rotation. We use this effect to design a controller that regulates pendulum motion based on the sensors in our system. To start, we must create a mathematical model of our system. Although the dynamics of the pendulum and reaction wheel are governed by angular acceleration, inertia, and applied torque, our sensors only provide angle measurements, so we model the system using 2nd order differential equations, plug in known parameters, and solve for the unknowns. To create our model in practice, we used the Lagrange method. We then created several iterations of negative feedback controllers to practice using different methods of control and refine our stabilization method. This report details our process of making and refining a system model, as well as iteratively designing controllers to stabilize our system.

Mathematical Model

$$\theta_p'' + \omega_{np}^2 sin\theta_p = -\frac{k}{J} i$$

$$\theta_r'' = \frac{k}{J} i$$

θ_p and θ_r are pendulum and rotor angle, while ω_{np} is the natural frequency of the pendulum around the pivot.

This set of equations illustrates how torque on the rotor $\tau = ki$ generated by a current i governs the angular acceleration of θ_p and θ_r by a transducer factor J and torque constant k.

Using the small angle approximation $sin(\pi + \delta\theta_p) \approx -\delta\theta_p$, our linearized state space model at upright ($\theta_p = \pi$) is as follows:

$$\begin{pmatrix} \delta\theta_p' \\ \theta_p'' \\ \delta\theta_r' \\ \theta_r'' \end{pmatrix} = \begin{pmatrix} 0 & 1 & 0 & 0 \\ a & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{pmatrix} * \begin{pmatrix} \delta\theta_p \\ \theta_p' \\ \delta\theta_r \\ \theta_r' \end{pmatrix} + \begin{pmatrix} 0 \\ -b_p \\ 0 \\ b_r \end{pmatrix} * u, \text{ where } \begin{matrix} \delta\theta_p = \theta_p - \pi \\ a = \omega_{np}^2 \\ b_p = \frac{k_u}{J}, b_r = \frac{k_u}{J_r} \end{matrix}$$

Developing PD Control

We began designing our feedback controllers by identifying key elements of our system using *closed-loop system identification*. We determined static and viscous friction by running our motor at a few constant reference speeds, measuring the amount of control needed, and using a polynomial fit to fit the resulting data and get values for each type of friction. This works because, at steady state, the wheel's angular acceleration is zero ($\dot{\omega}_r = \ddot{\theta}_r = b_r\left(u + F\left(\dot{\theta}_r\right)\right) = 0$), and thus has a control effort (motor torque) equal to the wheel's friction. Therefore, the control input required at each speed can be used to infer friction force. We chose to neglect friction in the pendulum arm, as it uses a bearing to spin, which we believe is efficient and results in negligible friction. We then analyzed the natural frequency of the pendulum ω_{np} by lifting it and measuring the frequency at which it swings. Then, we had to determine the torque constant of the motor by solving for b_r and b_p in our system. A naïve approach is to measure θ_p and θ_r, differentiate our values twice to get all the necessarily values, and plug and chug to solve for b_r and b_p. Unfortunately, our angular data is very noisy and results in increasingly meaningless first and second derivatives. Instead, we can find a polynomial fit to our data to get the smoothness we need, then differentiate those values to find our torque constants. In practice, we used a cubic fit to obtain the necessary differentiability and used classroom-given values of $b_r = 198$ and $b_p = 1.08$ because they were close enough to what we found.

After system identification, we began designing controllers to stabilize our system. We first designed a 2-state feedback system, where we just controlled the pendulum angle. We then added control for the wheel velocity so that the control would be limited and converge to a steady state, which made our controller a 3-state feedback system. Also added friction compensation, which is just $b\omega + c$ to the signal after the controller. Friction compensation was beneficial: it made our system converge to steady state faster. We think this is because, with friction compensation, the controller was able to deal with exclusively the non-friction forces, which allowed for finer tuning, instead of having to combat friction too.

Finding Equilibrium

$$x = \begin{pmatrix} \delta\theta_p \\ \theta_p' \\ \delta\theta_r \\ \theta_r' \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \\ \delta\theta_r \\ \theta_r' \end{pmatrix}, u = 0 \rightarrow x' = \begin{pmatrix} 0 & 1 & 0 & 0 \\ a & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{pmatrix} * x = \begin{pmatrix} 0 \\ 0 \\ \theta_r' \\ 0 \end{pmatrix}$$

Thus, the system is stable at equilibrium point. Since full state feedback is adopted,

$u = -Kx$

$x' = Ax - BKx$

$= x' = (A - BK)x$

Since the three states we care about are pendulum angle, pendulum speed, and rotor speed, our goal is to make $\delta\theta_p, \theta_p', and\ \theta_r'$ converge to 0.

```
bp_max = 0.95671;
br_max = 172.0067;

A = [0,1,0,0; a,0,0,0; 0,0,0,1; 0,0,0,0];
B = [0; -bp_max; 0; br_max];
p = [-5.28+7.04i,-5.28-7.04i, 0, -5.28];
K = place(A,B,p);

Output:
> K = [-218.824, -22.1688, 0, -0.0312]
```

This K value will make $\delta\theta_p$ converge to 0, and thus θ_p converge to π.

	Two-State Feedback		Three-State Feedback	Observer		
Max IC deviations	0.1225 $\delta\theta_p$	0.9 $\delta\dot{\theta}_p$	0.13 $\delta\theta_p$	0.95 $\delta\dot{\theta}_p$	0.0637 $\delta\theta_p$	0.55 $\delta\dot{\theta}_p$
Max pulse	4		6	3.7		
Max disturbance	4		7	4.5		

Table 1

Under two-state feedback, the model runs fine with only initial conditions, but for max pulse and max disturbance, the motor undergoes a constant acceleration to counteract, regardless of however small the disturbance is.

Under three state feedback, a pole is placed in LHS for θ_r' so that it will converge to 0 too, and thus, the angular velocity will not attempt to reach infinity the way two state feedback did.

Also, our initial code only stabilized the pendulum when it was rotated counterclockwise because of a sensor output issue. The controller was designed to stabilize at $\theta_p = \pi$ but the sensor reported absolute angle over an unbounded range, so π corresponded to a half turn CCW and additional CCW rotation continued increasing the measured angle. As a result, the controller would always want to spin the reaction wheel CW to return to "π", even when intended motion was CCW spinning. To fix this, we wrapped the pendulum angle signal modulo 2π. This constrained the measurement to the range $[0, 2\pi)$, allowing the pendulum to stabilize at π regardless of its initial direction.

Full State Feedback Control with Decoupled Observer

Observers are used when not all states are obtainable; they estimate the full state based on input and observed output. The reason why the observer can decouple is that $\delta\theta_p, \theta_p'$ are independent from $\delta\theta_r, \theta_r'$, as can be seen in our state space equation above. Also, some entries in L do not affect pole placement, but they can amplify noise and error in practice, which is why a two-state observer is advantageous.

Taken from lab manual:

2. Let's show that the error between the actual states and estimated states $\boldsymbol{e} = \boldsymbol{x} - \hat{\boldsymbol{x}}$ goes to zero over time.

First, we differentiate the equation to find error.

$$\boldsymbol{e'} = \boldsymbol{x'} - \boldsymbol{x\hat{}'}$$

Next, we prove that the poles are stable. Set $\boldsymbol{e'}$ equal to zero and show that $\boldsymbol{e}$ = [0,0,0,0] is the *only* stable equilibrium for this equation.

We substitute the equations we have for $\boldsymbol{x'}$, $\boldsymbol{x\hat{}'}$, and $\boldsymbol{u}$. $\boldsymbol{e}$ is the only variable.

$$e' = Ax + Bu - (Ax\hat{} + Bu\ + L(y - Cx\hat{}))$$

$$e' = A(x - x\hat{}) - LC * (x - x\hat{})$$

$$e' = (A - LC) * (x - x\hat{})$$

$$e' = (A - LC) * e$$

If $\boldsymbol{e'} = \boldsymbol{0}$ and we know $\boldsymbol{A - LC}$ is stable and full rank, $\boldsymbol{e}$ = [0,0,0,0]. Since the goal is to make $\boldsymbol{x - x\hat{}}$ converge to 0 much faster than the system itself does, the poles for $\boldsymbol{A - LC}$ need to be set much further in the LHS of the complex plane than the original system.

```
% eigenvalues of A-LC are poles of system

A = [0,1,0,0; a,0,0,0; 0,0,0,1; 0,0,0,0];
B = [0; -bp_max; 0; br_max];
C = [1 0 0 0; 0 0 1 0];
D = zeros(4,3)

wn = 10;
zeta = 0.5;

sys = tf([wn^2], [1  2*zeta*wn  wn^2]);
poles_new = pole(sys);

K_new = place(A, B, [poles_new(1) poles_new(2) 0 sum(poles_new)/2]);

A12 = [0 1; a 0];
B12 = [0; -bp_max];
C12 = [1 0];

p12 = [(-5*30)+7.04i,(-5*30)-7.04i];
% p12 = [real(poles_new(1))*7 + imag(poles_new(1))*j/30, real(poles_new(1))*7 +
imag(poles_new(2))*j/30];
L12 = place(A12', C12', p12')';


A34 = [0 1; 0 0];
B34 = [0; br_max];
C34 = [1 0];
p34 = [-50, -40];
% p34 = [sum(poles_new)/2*6, sum(poles_new)/2*7];
```

```
L34 = place(A34', C34', p34')';

L_new = [L12, [0; 0]; [0; 0], L34];
% C_new = [C12, 0, 0; 0, 0, C34]
% D_new = zeros(2,3)
eig(A-L_new*C)
% eig(A)
% eig(A12)
% eig(A34)
```

Verification:

```
eig(A-L_new*C):

ans =

  1.0e+02 *

 -1.5000 + 0.0704i

 -1.5000 - 0.0704i

 -0.5000 + 0.0000i

 -0.4000 + 0.0000i
```

3. Values listed in **Table 1** show the maximum IC deviation, pulse disturbance, and constant perturbation that the controller can stabilize from the Simulink simulations of the RWP.

4. The behavior of the controller with the observer tends to be more sensitive as it can only handle a smaller range of IC, max pulse, and max disturbance compared to full state feedback controller. This does make sense because all the states are being estimated instead of being directly observed.

Conclusions

Judging by range of IC deviation, max pulse, and max disturbance, full state feedback controller is better as it can take in a larger range of them and still can bring the system back to stability. On the other hand, if the states are not easily obtainable, the observer definitely wins out since there is no way to implement full state feedback controller.

Part 6 *(Extra credit)*

Part 6 of our project required us to implement *switching control*: alternating controllers based on pendulum position. Since the RWP has two equilibria, upward and downward, we can choose to stabilize at either equilibrium based on the initial position of the pendulum. If the pendulum is placed at the upward position, it should stabilize upward, and if it is placed downward, it should stabilize downward. We re-implemented our previous state-feedback controller by adding two switches with conditions for each case. We decided to detect when the pendulum is facing up or down by taking the cosine of the $mod(2\pi)$ pendulum angle so that our switch gives us an “up controller” when the angle is positive and a “down controller” when it is negative.

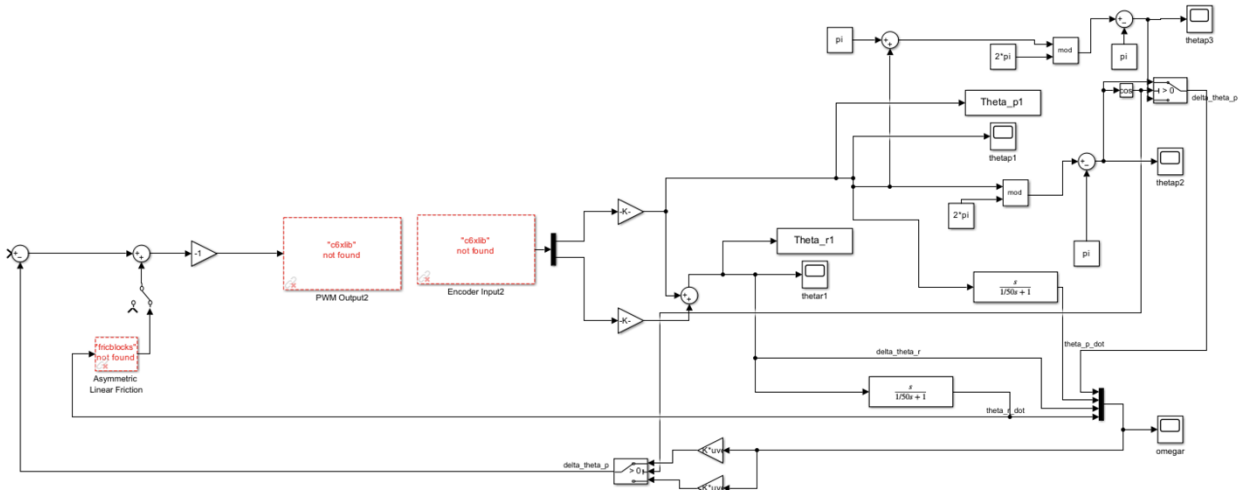
Our up controller has the same behavior as the normal inverted stabilizing controller we already had, so we didn't need to make anything new. However, to stabilize down, we had to design a down controller. For this new case, we had to change the reference angle of the controller to 0 rad instead of π, and we also had to re-linearize our system to make the controller spin the correct way to stabilize at the bottom. This only involved adding a negative sign in our A matrix to the linearly approximated a term. Our end behavior resulted in the controller stabilizing to the down position if it was held below the horizontal axis and stabilizing to the inverted position if held above the horizontal axis.

Part 7 *(Extra credit)*

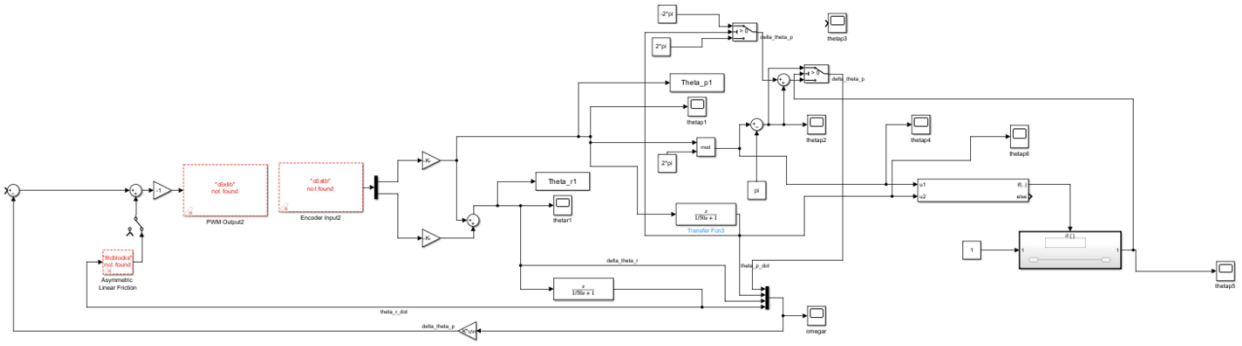
Part 7 of the project required us to make a swing-up controller. We previously made a controller that does this behavior in part 4, but it was very slow. The reason why our part 4 controller worked at all was because of the strength of the motor. Simply, our part 4 controller calculated the shortest distance towards the upward position and then tried to always move the pendulum in that direction. So, if our pendulum was on the right side of the setup, it would always try to spin up to the right, and vice versa for the left side. If our motor was able to spin the reaction wheel fast enough to counteract gravity, it would have worked in one swing, but our motor was not that strong. Therefore, what happened was our controller would spin the rotor a little bit in one direction, then gravity would overpower it despite the motor trying to go the other way, and the pendulum would then gain a little momentum towards the opposite direction on the down swing while still spinning the rotor the wrong way. Then at the very bottom, it would switch rotor spin direction because the pathway now became shorter in the opposite direction, and this process would continue until eventually, after around 50 seconds, the pendulum would swing up. This is obviously very inefficient, so we decided to develop a new method.

After a few rounds of testing, the method we settled on was this: spin the rotor one way, detect when the pendulum stops moving (the inflection point of $\dot{\theta}_p$ where it becomes 0 and swaps sign), and then spin the rotor the other way at the top of the swing. We implemented this by creating an *if* statement subroutine in our Simulink controller that detects which direction the wheel is spinning per side, detects which way it should be spinning based on the sign of $\dot{\theta}_p$, and then spins it properly. Initially, we used the down controller from part 6 to make it spin downwards at the top of the swing, but we realized that this wasn't as efficient as possible because the down controller was "trying" to stabilize at the bottom, not swing down so that the pendulum can gain momentum up to the top. So, we solved this by doing a coordinate transformation: using the property that our controller will always try to take the shortest path towards the top, we added or subtracted 2π based on our current position when $\dot{\theta}_p$ changes sign. If it's on the left, we will subtract 2π to turn our current coordinate from $\sim\frac{\pi}{4}$ to $(-2\pi + \sim\frac{\pi}{4})$, and vice versa on the right side. This makes our controller "try" to swing up the opposite direction once it reaches the inflection point of $\dot{\theta}_p$ because that is now the shortest distance to pi. This method worked extremely well compared to our old method, and it made our swing-up controller fully stabilize on average in 9 swings, down from nearly 80.

Appendix



Part 6 Controller



Part 7 Controller